

SECTION 4:

What should be the return type of this method?

Ans:

Kiểu trả về của phương thức toString() trong Java luôn phải là String.

Phương thức này được dùng để ghi đè (override) phương thức toString() mặc định của lớp Object, nhằm trả về chuỗi mô tả đại diện cho đối tượng.

SECTION 13:

Which classes are aggregates of other classes?

Ans:

- CompactDisc là aggregate của Track vì một CD chứa danh sách các đối tượng Track.
- Cart là aggregate của Media vì giỏ hàng quản lý một tập hợp các sản phẩm Media.
- Store cũng là aggregate của Media do cửa hàng lưu trữ danh sách các sản phẩm thuộc kiểu Media.

SECTION 15:

If the passing object is not an instance of Media, what happens?

Ans:

- **Nếu có sử dụng instanceof:**

Phương thức sẽ trả về false.

Cách này giúp chương trình an toàn hơn, tránh lỗi NullPointerException (khi đối tượng là null) và đảm bảo tính hợp lý vì hai đối tượng khác kiểu không thể được xem là bằng nhau.

- **Nếu không kiểm tra instanceof mà ép kiểu trực tiếp:**

Chương trình sẽ phát sinh ngoại lệ ClassCastException, dẫn đến lỗi khi chạy (runtime error).

Kết luận:

Việc kiểm tra kiểu dữ liệu trước khi ép kiểu là cần thiết để tăng độ ổn định của chương trình và tuân thủ nguyên tắc lập trình hướng đối tượng trong Java.

SECTION 16:

1. What are the advantages of Polymorphism?

Ans:

- **Khả năng mở rộng (Extensibility):**

Có thể thêm các lớp con mới (ví dụ Magazine) mà không cần sửa đổi code cũ. Hệ thống vẫn hoạt động bình thường.

- **Tái sử dụng mã nguồn (Reusability):**

Chỉ cần viết một đoạn xử lý cho lớp cha Media, các lớp con đều có thể sử dụng lại.

- **Giảm sự phụ thuộc (Loose Coupling):**

Giúp tách biệt giữa lời gọi phương thức và cách phương thức được thực thi, làm hệ thống linh hoạt hơn.

- **Đơn giản hóa chương trình:**

Không cần dùng quá nhiều if-else hay switch-case để kiểm tra kiểu đối tượng. Chỉ cần gọi: m.toString();

2. How is Inheritance useful to achieve Polymorphism in Java?

Ans:

- **Tạo quan hệ “IS-A”:**

Kế thừa xây dựng mối quan hệ phân cấp giữa các lớp. Ví dụ: CompactDisc là một Media.

- **Hỗ trợ ghi đè phương thức (Overriding):**

Nhờ kế thừa, lớp con có thể định nghĩa lại hành vi của lớp cha để tạo ra đa hình.

- **Hỗ trợ ép kiểu lên (Upcasting):**

Có thể xem đối tượng lớp con như đối tượng lớp cha, ví dụ:

Media m = new Book();

Đây là nền tảng quan trọng để thực hiện đa hình trong Java.

3. What are the differences between Polymorphism and Inheritance in Java?

Ans:

Về bản chất:

- **Kế thừa (Inheritance):**

Là cơ chế cho phép lớp con thừa hưởng thuộc tính và phương thức từ lớp cha. Nó tạo ra quan hệ “IS-A”.

- **Đa hình (Polymorphism):**

Là khả năng nhiều đối tượng phản hồi cùng một phương thức theo những cách khác nhau. Ví dụ:

Cùng gọi toString() nhưng Book và CD sẽ hiển thị nội dung khác nhau.

Về mục đích:

- **Kế thừa:**

Tập trung vào tái sử dụng mã nguồn.

- **Đa hình:**

Tập trung vào tính linh hoạt và khả năng mở rộng của hệ thống.

Về thời điểm xác định:

- **Kế thừa:**

Được xác định ở thời điểm biên dịch thông qua từ khóa extends.

- **Đa hình:**

Thường được xác định khi chương trình chạy (runtime), Java sẽ quyết định phương thức nào được gọi dựa trên đối tượng thực tế.

Về mối quan hệ:

- Kế thừa là nền tảng để tạo ra đa hình.

- Đa hình là kết quả của kế thừa kết hợp với cơ chế ghi đè (@Override).

Về khái niệm liên quan:

- **Kế thừa:** extends, super, base class, derived class.

- **Đa hình:** overriding, overloading, dynamic binding, upcasting.

SECTION 17:

1. What class should implement the Comparable interface?

Ans:

Lớp Media nên implement Comparable<Media>. **Giải thích:**

Vì Cart chứa danh sách các đối tượng Media, nên để sử dụng:

`Collections.sort(itemsOrdered);`

thì các đối tượng trong danh sách phải tự biết cách so sánh với nhau thông qua phương thức `compareTo()`.

2. How should you implement the compareTo() method?

Ans:

Bước 1: So sánh theo title

Dùng:

`compareToIgnoreCase()`

để so sánh tiêu đề mà không phân biệt chữ hoa hay chữ thường.

Bước 2: Kiểm tra kết quả

- Nếu kết quả khác 0 → trả về ngay.
- Nếu bằng 0 → tiếp tục so sánh thuộc tính khác.

Bước 3: So sánh theo cost

Dùng:

`Float.compare(other.cost, this.cost)`

để sắp xếp giá theo thứ tự giảm dần (giá cao hơn đứng trước).

Bước 4: Trả về kết quả

- Số âm → đối tượng hiện tại đứng trước.
- Số dương → đối tượng hiện tại đứng sau.
- 0 → hai đối tượng ngang nhau.

3. Can we have two ordering rules if using Comparable?

Ans:

Không thể. Giải thích:

Comparable chỉ cho phép định nghĩa duy nhất một phương thức `compareTo()`, tức chỉ có một “natural ordering”.

Nếu muốn hỗ trợ nhiều cách sắp xếp như:

- title → cost
- cost → title

thì cần sử dụng Comparator.

4. Suppose DVDs have a different ordering rule. How would you modify the code?

Ans:

Ý 1: So sánh thuộc tính chung (title)

Trước tiên, so sánh title của hai đối tượng Media.

Nếu khác nhau → trả về kết quả luôn.

Ý 2: Kiểm tra kiểu dữ liệu

Nếu tiêu đề giống nhau, dùng `instanceof` để kiểm tra xem cả hai có phải là `DigitalVideoDisc` hay không.

Điều này giúp áp dụng logic riêng cho DVD mà không ảnh hưởng tới Book hoặc CD.

Ý 3: So sánh thuộc tính riêng của DVD (length)

Sau khi ép kiểu về DigitalVideoDisc, tiến hành so sánh length theo thứ tự giảm dần.

DVD dài hơn sẽ được xếp trước.

Ý 4: So sánh cost

Nếu:

- hai DVD có cùng title và length,
 - hoặc hai đối tượng không phải DVD,
- thì tiếp tục so sánh cost theo thứ tự giảm dần để xác định kết quả cuối cùng.